

Spacecraft Scheduling in Resource-Constrained Environment using Proximal Policy Optimization

Michael Plano and Logan Walthers

Abstract—We discuss the design and implementation of a Proximal Policy Optimization algorithm as applied to autonomous satellite scheduling and commanding. The scheduling problem is formulated as a Partially Observable Markov Decision Process in which the policy maker, a PPO, must balance the mission of imaging city targets with maintaining its limited battery life under realistic slew and atmospheric drag constraints. The simulation is built using the Basilisk Astrodynamics package and the BSK-RL reinforcement learning library. We define 1000 city targets with priority weighted rewards and a terminal battery failure condition. The PPO was then defined from first principles with PyTorch using a shared actor-critic network, Generalized Advantage Estimate, and a clipped surrogate objective. The PPO successfully learned non-myopic tradeoffs and achieved a higher reward compared to the baseline policies.

I. INTRODUCTION

Autonomous spacecraft scheduling is one of the largest problems in the aerospace industry today. Satellite constellations have grown in scale and complexity, leading to an increase in operation cost. Ground-based command sequences are challenged by communications windows and constantly evolving orbital dynamics, reducing opportunities for payload operation to short durations. Waiting for an operator to respond to real-time events can cause satellites to miss opportunities critical to their mission, motivating development of autonomous decision-making systems onboard the spacecraft. These decision-making agents must conform to the constraints of the system while simultaneously executing the mission.

Reinforcement Learning techniques offer a framework for developing autonomous spacecraft controls. Spacecraft tasking and scheduling can be formulated as a Partially-Observable Markov Decision Process (POMDP), and an agent can learn a policy for taking actions depending on its observation of the system state [1]. System parameters such as orbital position, battery charge, and opportunity windows can be read as states and mapped to actions such as imaging specific targets or charging the battery. Reinforcement Learning (RL) methods can learn from the environment, or in this case, a physics-based simulation environment. The resulting policies learned by RL algorithms can capture complex behavior that are difficult to encode using traditional optimization techniques. [2]. Recent developments in RL methods have led to the Proximal Policy Optimization (PPO) algorithm, which has become widely adopted in many fields due to its stability and ease of implementation [2].

In this paper, we discuss the design and implementation of a custom PPO agent for scheduling an Earth-observation satellite focused on imaging cities from Low-Earth Orbit. The

PPO is an on-policy actor-critic algorithm that learns a strategy and value function to estimate mission returns. It is built from scratch using PyTorch and includes Generalized Advantage Estimation (GAE) for variance reduction and a clipped surrogate objective to stabilize policy updates [3]. The spacecraft environment is simulated using the Basilisk Astrodynamics Framework and tools from the BSK-RL package to create a model of orbital mechanics, attitude dynamics, and the power subsystem [1]. The mission scenario includes 1000 city targets with assigned priority weightings.

We compare the trained PPO policy with two baselines: a random policy and a heuristic policy. All strategies are evaluated across four metrics: total accumulated reward, imaging efficiency, number of unique cities imaged, and episode length. We identify three levels of success for this exercise as follows:

-Level 1: Train an off-the-shelf PPO agent on a prebuilt simulation and reward function. To reduce complexity, a simplified satellite model was used. This spacecraft model assumed unlimited power, constant slew, and a Keplerian orbit. Success was defined as achieving improved reward compared to the random baseline policy.

-Level 2: We extend the mission by adding constraints to more accurately model a real-world system. These constraints include limited battery capacity, slew rate constraints, and atmospheric drag. In addition, we introduce complexity in our reward function by defining a priority weighing system to distinguish high- and low-value targets. Success was defined as achieving an improved reward compared to both the random and domain-informed heuristic baseline policies. Performance was evaluated using the total cumulative priority-weighted reward at the conclusion of the simulation.

-Level 3: A further increase to mission realism includes moving targets, communication scheduling, data downlink windows, data buffer management, resilience to inconsistent target location information, and constellation scheduling. We define success in this scenario as maximizing the cumulative priority-weighted reward and maximizing the median image collection time per target while avoiding spacecraft failure events across all satellites in the satellite formation.

II. BACKGROUND & RELATED WORK

The problem of scheduling Earth-orbiting satellites has been studied for many years as an optimization problem. Traditional approaches have included techniques such as linear and constraint programming, genetic algorithms, and simulated annealing [1]. These methods have had some success, but are not scalable given the increasing number of satellites

and resource constraints, and are generally not adaptable to changes in environmental conditions. Issues such as hardware malfunctions, weather, and changing mission priorities can shift operational decision making and make static policies obsolete. This motivates the shift toward learned policies that are lightweight and can solve new, novel optimization problems quickly [4].

A. Reinforcement Learning for Satellite Scheduling

It has been shown that satellite scheduling can be successfully formulated as a POMDP problem that can be handled by RL algorithms [5]. Reinforcement learning techniques have become a popular field of study with the advancement of machine learning in recent years, and have had some success in spacecraft scheduling exercises. In their 2023 paper, Herrman and Schaub cover a variety of successes, including the application of RL algorithms to resource-constrained satellite scheduling [2]. Their work particularly shows that the PPO implemented as an actor-critic with clipped surrogate outperforms several other techniques. RL techniques have also been shown to be effective for spacecraft constellations [6].

B. PPO and Actor-Critic

The Proximal Policy Optimizer was proposed as an improvement upon the Trust Region Policy Optimizer (TRPO) [3], [7]. The TRPO constrains policy updates using a KL divergence bound computed from a second-order optimization step, whereas the PPO uses a clipping mechanism that reduces computation cost while achieving similar results. The clipped surrogate caps the large policy updates that would move the probability ratio outside of some interval, preventing policy collapse that occurs in unconstrained gradient ascent. The policy update is clipped to prevent large policy changes between episodes, which creates a more stable, "smoothed" policy change over time, versus implementing large policy updates that can potentially be disruptive [3].

The PPO uses two function approximations, the policy network (the actor), and the value network (the critic). The actor maps the state to an action probability distribution and the critic estimates expected discounted return from possible states. The critics value estimates are used for Generalized Advantage Estimation to calculate advantage for policy updates [8]. The GAE estimate balances the high variance Monte Carlo return and the one-step biased Temporal Difference residual through a λ factor that allows control over the bias/variance balance [8]. The PPO collects experience from rollouts on a working policy, then updates via stochastic gradient descent [3].

C. Basilisk and BSK-RL

The Basilisk package in python is an astrodynamics framework for modular simulation of spacecraft systems [9]. The package provides models of attitude dynamics, orbital mechanics, satellite subsystems, and flight software, and has been validated with hardware experiments and real flight data [9]. BSK-RL is a wrapper for Basilisk with a learning interface to

the Gymnasium package in python, allowing Reinforcement Learning methods to interact with the Basilisk Simulation. The BSK-RL package has several modules, including satellite models, city targeting scenarios, state-observation mappings, and reward functions [1]. It is possible to create a variety of simulations using the modular and extensible framework of this package.

This exercise makes use of the SteeringImager flight software model for calculating attitude and angular rate constraints, and the CityTargets module for populating city targets to model real-world locations. We use a priority-weighted scheduler to assign priority weights to different cities to force our learning model to discriminate between high- and low-value targets. The imaging cost/reward is coupled with a battery management objective to balance power consumed by tasks against charging maneuvers and heavily disincentivizes battery failure.

III. PROBLEM FORMULATION

We formulate the autonomous satellite tasking problem as a Partially-Observable Markov Decision Process (POMDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, \mathcal{R}, \gamma)$. In this definition, $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{O}$ is the observation space, $\mathcal{T}(s_{t+1} | s_t, a_t)$ represents the transitions, $\mathcal{R}(s_t, a_t)$ is the reward function, and $\gamma = 0.99$ is the discount factor.

The true $\mathcal{S}$ includes the dynamics of the spacecraft and the environment, including the position, velocity, attitude, and angular rates of the spacecraft body. The battery state and future collection opportunities are included in the state as well. However, the agent does not have access to the full state and instead operates on a partial observation, $o_t \in \mathcal{O}$.

At each step t , the agent selects an action $a_t \in \mathcal{A}$ according to the policy $\pi(a_t | o_t)$. This policy has the goal of maximizing the expected reward defined in the following equation:

$$\mathbb{E} \left[\sum_{t=0}^T \gamma^t \cdot \mathcal{R}(s_t, a_t) \right] \quad (1)$$

A. Observation Space

The observation o_t is a representation of the spacecrafts internal state. This state is limited by fixed parameters and limited lookahead for future opportunities. This observation includes the battery charge fraction ($b_t \in [0, 1]$), environmental information like the time until the next eclipse entry and exit, as well as the next 5 collection opportunities. Each collection opportunity includes the target priority, viewing angle, time until access window opens, and time until access window closes. All temporal quantities are expressed in relative simulation time, and all measurements are normalized to stabilize training.

B. Action Space

The action space is defined as containing 6 discrete options $\mathcal{A} \in \{0, 1, 2, 3, 4, 5\}$. Action $a = 0$ is a charging action that prioritizes battery accumulation. Actions $a \in \{1, 2, 3, 4, 5\}$ selects one of the next five available target opportunities

for imaging. Imaging actions are ordered corresponding to their upcoming target access opening windows. For example, selecting $a = 2$ means choosing to pass the first opportunity and image the second opportunity on this step.

C. Transition Dynamics

The transition function $\mathcal{T}(s_{t+1} \mid s_t, a_t)$ is governed by a high-fidelity physics-based simulation using the Basilisk framework [1]. This framework includes orbital propagation, access computation, attitude dynamics, pointing constraints, power consumption, and battery charging dynamics. In this scenario, the transition dynamics are trusted to the Basilisk library and are therefore treated as a black-box.

D. Reward Function

The reward function is designed to reflect mission value through successful image collection. At each timestep, the agent receives reward according to the following equation:

$$\mathcal{R}(s_t, a_t) = \begin{cases} w_i, & \text{if unique image of target} \\ -10, & \text{if battery depleted} \\ 0, & \text{o.w.} \end{cases} \quad (2)$$

where $w_i \in [0, 1]$ represents the priority of target i .

This function only provides reward upon successful imaging of a unique target. It also weights higher value targets with more rewards. Additionally, terminal events such as reaching the time limit or running out of battery prevents further reward accumulation.

E. Episode Termination

Episodes are terminated under one of the conditions listed below

- 1) Maximum time horizon has been reached
- 2) Maximum episode step count has been reached
- 3) The spacecraft battery has reached a depleted state
- 4) Some other terminal event triggered

F. Objective

The objective is to learn a policy $\pi(a_t \mid o_t)$ that maximizes the total mission value while conforming to constraints. The agent must learn to prioritize high-value targets, schedule actions with limited future visibility, manage battery resources, and avoid failure states. This creates a constrained decision-making problem.

G. Environment and Scenario Initialization

To evaluate the proposed approach, we define a specific scenario using the BSK-RL framework

1) *Orbit Configuration*: The spacecraft was placed in a low-earth orbit (LEO) with a constant altitude of 500 km for 5 hours. The inclination, longitude of the ascending node, argument of perigee, and true anomaly were all randomized. This orbit is initialized as Keplerian, but due to the high-fidelity propagation, the trajectory will get perturbed as the simulation progresses.

2) *Target Set*: The mission consists of a set of 1000 real-world city targets generated using the CityTargets method provided by BSK-RL. These targets are geographically distributed locations on Earth and are imaging opportunities for the spacecraft. Each target is assigned a uniform random scalar priority $w_i \in [0, 1]$ according to the BSK-RL library.

3) *Sensor Model*: The spacecraft has a single sensor on board. For simplicity, the sensor was modeled with an extremely narrow field-of-view. This means that any target off bore-sight will not be considered in the reward function. No incidental collection will accumulate rewards.

IV. SOLUTION APPROACH

In order to best learn a policy for our satellite simulation, we use a PPO implemented from first principles using PyTorch. The PPO is chosen because other policy gradient methods can have large, destructive policy updates that can cause the overall policy to fail, and because it is shown to consistently converge to an optimal policy in the BSK-RL environment [2]. The PPO components are described below:

A. Actor-Critic Network Architecture

The policy and value functions for the PPO are implemented using a shared actor-critic neural network as a feedforward multilayer-perceptron (MLP). It has two fully-connected hidden layers with 64 nodes each and uses ReLU activation to extract shared features. It maps the 24-dimensions of the observation space to a higher 64-dimensional space and shares parameters between policy and value functions to learn state representations for action selection and value estimation [10].

The actor head decides its action based on state, taking the 64-dimensional feature from the trunk and producing normalized scores for each possible action representing the network's preference for that action given the current state. When training, an action is sampled from a categorical distribution for exploration, and during evaluation the argmax of all values is taken [10].

The critic head makes an estimate of the current state, taking in the same 64-dimensional feature and producing a single scalar state-value, $V(s_t)$, an estimate of the total discounted reward from that point forward, given the current state and actions possible, with a high value representing a better reward. The critic head does NOT make decisions, instead only calculating advantage for the training, telling the policy whether its action performed better or worse than expected [10].

B. Generalized-Advantage Estimate

During the critic advantage estimate, we use the Generalized Advantage Estimate (GAE) to calculate advantage. GAE provides balance between a high-variance Monte Carlo estimation and the biased temporal difference target estimation [8]. GAE is calculated in a single reverse pass over the rollout buffer using the equation

$$\hat{A}_t = \sum_{t=0}^{T-t} (\gamma\lambda)^1 \cdot \delta_{t+1} \quad (3)$$

where $\gamma = 0.99$ is the discount factor and $\lambda = 0.95$ is the smoothing parameter. The critic is trained using $G_t = \hat{A}_t + V(s_t)$. Tuning of λ allows favoring variance reduction for when episode length is variable due to terminating on battery failure events [8]. The advantages are also normalized to zero mean and unit variance per minibatch to stabilize gradient update magnitude [3]. At the end of the critic run, a bootstrap value is calculated from the existing samples to handle incomplete episodes where the rollout halted due to step and episode caps if they are hit.

C. Clipped Surrogate

We update the policy by using a clipped surrogate loss function and maximizing its output. The clipped surrogate policy loss is defined as:

$$L_{clip} = E_t[\min(\rho_t \cdot \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon_{clip}, 1 + \epsilon_{clip}) \cdot \hat{A}_t)] \quad (4)$$

Where we let the clip parameter $\epsilon_{clip} = 0.2$. This type of clipping function prevents the policy from moving outside the interval $[1 - \epsilon_{clip}, 1 + \epsilon_{clip}]$ on each update, creating a bound on the update. This cap is put in place to prevent destructive updates to the policy, which could potentially cause the policy to collapse from too large of a jump in values during the update epochs [3]. We attempt to minimize mean-squared-error (MSE) loss between our predicted value and the GAE targets and use a clipped value loss to constrain the value update to within ϵ of the previous value estimate as follows:

$$L_{VF} = 0.5 \cdot E_t[\max((V_\theta(s_t) - G_t)^2, (\text{clip}(V_\theta(s_t), V_{old} \pm \epsilon_{clip}) - G_t)^2)] \quad (5)$$

We then combine the above terms to form the total training objective:

$$L(\theta) = -L_{clip}(\theta) + c_v \cdot L_{VF}(\theta) - c_H \cdot H(\pi_\theta) \quad (6)$$

where $c_v = 0.5$ is the value loss coefficient and $c_H = 0.01$ is the entropy bonus coefficient. There is also an entropy term defined in the equation below

$$H(\pi_\theta) = -\sum_a \pi_\theta(a|s) \log \pi_\theta(a|s) \quad (7)$$

Equation 7 describes the penalization of convergence to deterministic policies and encourages exploration while training [10]. Gradients are also clipped to the L2 norm of 0.5 before updates to prevent uncontrolled exponential growth [10].

D. Training

We run the training for 500 iterations, and at each iteration use the current policy to collect N steps across one or more episodes, letting $N = 2048$ environment steps with a max of 300 steps per episode. These steps are stored in a rollout buffer alongside value estimates and probabilities. We then compute GAE Advantage and value over the whole buffer, before shuffling it and splitting it into minibatches of 64 steps.

Ten epochs of gradient-descent updates are then performed over each minibatch using an Adam optimizer with a learning rate of $\alpha = 3 \times 10^{-4}$ and $\epsilon = 10^{-5}$. At intervals of 25 iterations, we save model weights, optimizer state and several hyper-parameters to a checkpoint file to allow for evaluation at several stages of the training. We also include a battery failure wrapper to detect the rate of battery failure events, a terminal state in our simulation with a large negative reward. Additionally, we track training entropy to detect ratio of exploration vs. exploitation of objectives throughout the policy training [2].

E. Evaluation

After training is done, we use our checkpoint files to run the learned policy using a greedy (argmax) algorithm with no stochastic sampling [2]. We also evaluate heuristic and random policies as baseline controls for comparison, given the same initial conditions. We evaluate mean total reward, number of unique cities imaged, priority-weighted imaging efficiency (total reward over number of cities imaged), and mean episode length

F. Miscellaneous Design Considerations

The BSK simulation uses a runtime error when the internal state of the simulation hits a battery failure state. Some logic was added around the error to treat it as a terminal state, and reset the environment with some seeded initialization values. This prevents the physics and modeling of Basilisk from crashing our training loop and allows the PPO to recognize the consequences of allowing battery failure [1].

We use a sparse, priority-weighted imaging reward to positively reinforce the imaging of unique cities. The entropy bonus c_H , GAE- λ , and stochastic action sampling in the training step were tuned to encourage city collection as opposed to converging to an ‘always charge’ strategy.

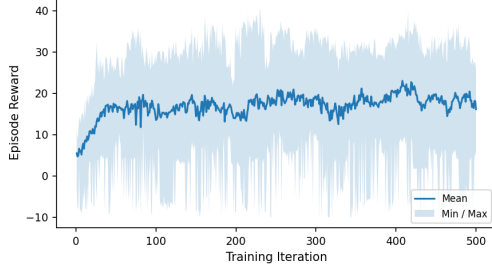
All observations were normalized before being passed into the PPO network. The target angles are normalized by $\pi/2$, opportunity windows to 300 seconds, and eclipse time to 5,700 seconds. This allows input features to be on comparable scales during training and evaluation [10].

V. RESULTS

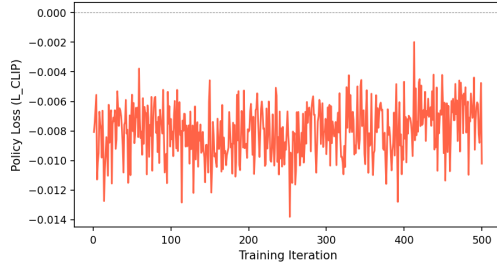
A. Training Performance

Given the training parameters provided above, the PPO was trained for eight hours of wall-clock time. Figure 1a shows the mean episode reward per training iteration over all iterations, showing improvement over the training period, increasing from a mean of approximately 5.5 at training start to 19.0 at its peak. The minimum episode reward reflects worse-case conditions in battery management and also trends upward, eventually becoming predominantly non-negative, showing the agent learning to avoid battery depletion. The clipped surrogate policy loss (Figure 1b) shows consistent negative values, indicating policy improvements over the previous iteration at each step. In Figure 1c we see the mean-squared error of value loss decreasing sharply before gradually increasing, consistent

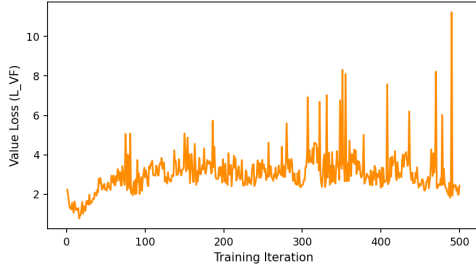
with the policy improving with training, with returns growing larger and the critic changing its estimate of the policy. Lastly, Figure 1d shows a marked downward trend in policy entropy as the learned policy converges and chooses exploitation of a deterministic strategy over exploration.



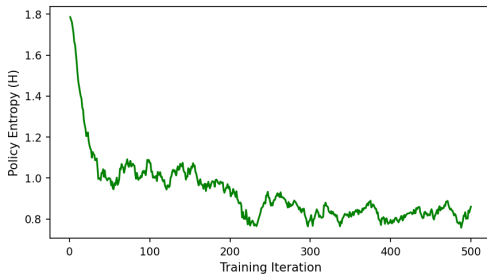
(a) Episode reward vs. Training Iteration. Mean reward is depicted by the solid line, and max/min reward are shown in the shaded region.



(b) Clipped Surrogate Policy Loss vs. Training Iteration. Negative values indicate policy assigning higher probability to advantageous actions.



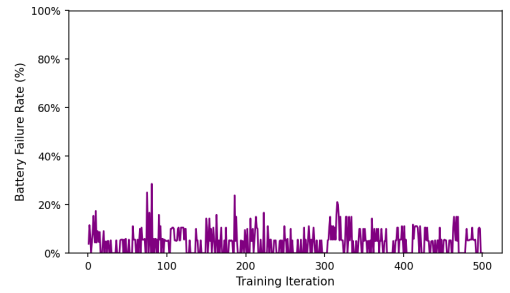
(c) Critic MSE Value Loss vs. Training Iteration. Value Loss decreases as critic learns coarse return estimate, then increases as policy improves



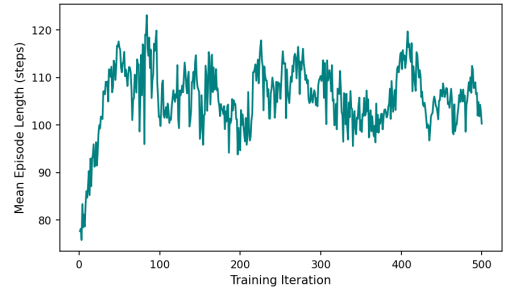
(d) Policy Entropy vs. Training Iteration. Policy Decrease reflects progressively more deterministic scheduling strategy.

Fig. 1: PPO training losses

The environment metrics in Figure 2a show a relatively stable, but low rate of battery failure over training. This could indicate the policy was either not trained well enough, or that negative rewarding was not sufficient to motivate our policy to take better care of the battery. However, due to the low initial rate, this flat trend was deemed acceptable for our simulation. The episode length in Figure 2b increases as training iterations go on. This length is bounded by both time limit and step count, and indicates our policy surviving longer in each episode, which is a direct consequence of better battery management practices being learned. The flat rate of battery failure rate combined with increased episode length indicates that the policy is learning how better to manage battery charge to keep the mission going longer, but not enough to completely avoid any risk of battery failure.



(a) Battery Failure Rate vs. Training Iteration



(b) Mean Episode Length vs. Training Iteration

Fig. 2: Training Environment Metrics. Battery Failure Rate does not decrease to mean of 0%, but mean episode length increases with training, showing that the agent is learning battery management behavior

B. Policy Evaluation

1) *Evaluation Setup:* To evaluate the performance of the trained PPO policy, we compare it against two baseline policies.

- 1) A policy that selects uniformly random actions
- 2) A heuristic policy that selects the highest-priority available target, unless the battery charge fraction is below 0.2, in which case it selects the charge action

All policies were evaluated over five episodes with identical initial conditions. During policy evaluation, PPO policy operates by selecting the action with the highest expected future reward. Each episode is simulated for a maximum duration of 5 hours, 100 steps, or until a terminal condition is reached. Performance is evaluated using the following metrics:

- Total accumulated reward
- Number of unique cities imaged
- Imaging efficiency - Reward / Number of Images
- Episode length

2) *Aggregate Performance Comparison:* Figure 3 highlights the difference in performance between the PPO, heuristic, and random policies using the metrics of total accumulated reward, number of unique cities imaged, and imaging efficiency.

This figure shows that the PPO significantly outperforms both baselines in terms of total reward and number of targets imaged, while the heuristic policy achieves the highest imaging efficiency. By achieving the highest total reward and number of cities imaged, the PPO policy showcases its ability to effectively balance target prioritization with resource constraints.

The heuristic policy achieves slightly higher imaging efficiency. This is intuitive given its design to always choose the highest priority target. This feature comes at the cost of total mission performance because the heuristic policy images significantly fewer total targets resulting in lower reward.

This distinction highlights the key difference between the PPO and heuristic approaches. The heuristic greedy policy maximizes immediate reward, while the PPO learns a policy that considers more long-term consequences. These future consequences consider resource management and future opportunities. Because of this approach, the PPO achieves superior mission performance despite lower action efficiency.

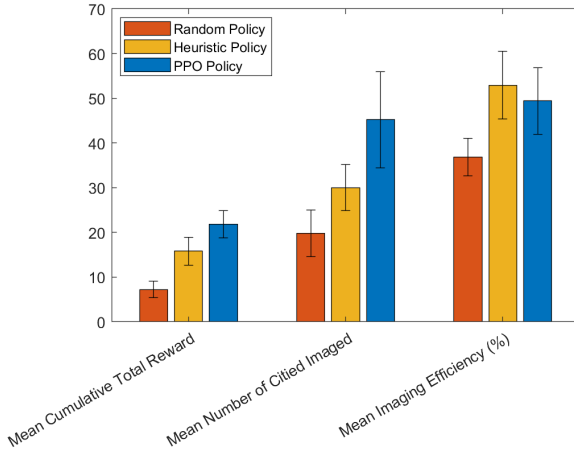


Fig. 3: Comparison of PPO Performance with Heuristic and Random Baselines. The bar heights are the mean values and the error bars indicate the standard deviation. The PPO achieved the highest cumulative reward and number of unique images. The heuristic policy achieved the highest imaging efficiency.

3) *Resource Management and Failure Behavior:* The PPO policy was also able to manage its onboard resources more effectively than both baselines. This becomes clear when comparing the episode length from the policies.

As shown in Table I, the PPO policy achieves longer episode durations and more frequently completes the full simulation horizon. The random policy episodes often end

Policy	Avg. Episode Len.	Std. Dev. Episode Len.
PPO Policy	99.00	1.549
Random Baseline	84.00	9.529
Heuristic Baseline	97.00	4.648

TABLE I: Mean and standard deviation of episode lengths for PPO, heuristic, and random policies. Longer episode duration indicates improved resource management and avoidance of failure conditions.

due to battery depletion. While the heuristic policy includes a check, this policy still sometimes fails due to insufficient resource management. The PPO policy is able to balance charging and imaging actions based on the current state and anticipated future states. This enables the mission to both avoid terminal states for longer, demonstrating the PPO is more reliable compared to the other baseline policies.

4) *Consistency Over Time:* Figure 4 shows the total reward as a function of episode step for the PPO, heuristic, and random policies.

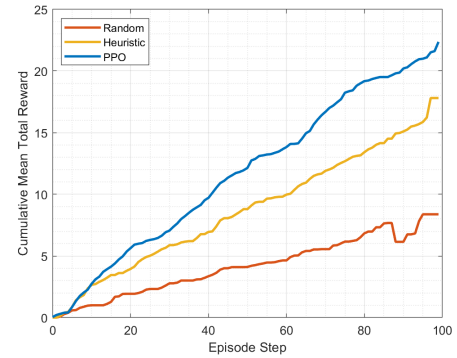


Fig. 4: Mean Cumulative Reward over Episode progress for PPO, heuristic, and random policies. The random policy achieves significantly worse rewards than both the PPO and heuristic. The PPO and heuristic policy start very similarly, then the heuristic reward rate slows and the PPO keeps growing at a constant rate.

Figure 4 shows that the PPO consistently accumulates reward faster than both the heuristic and random baselines. Both the PPO and heuristic policy achieve steady growth, however the PPO is likely able to achieve a faster rate due to exploiting the timing and sequence of opportunities. This figure reinforces that the PPO policy is not just achieving a higher final reward, but is achieving a higher reward consistently, through more effective decision making and opportunistic imaging.

5) *Shortfalls and Caveats:* There were several limitations of the work that warrant further investigation. The trained PPO policy outperformed the domain-informed heuristic baseline policy by approximately 35% averaged across our samples. This is due to the heuristic being well-formed, and the PPO having room for improvement. The heuristic is a competitive baseline for this problem, and the trained PPO must learn from reward signals alone to improve on its return. The PPO

would benefit from further training, larger network architecture, and reward shaping.

There is also some disparity between the target selected by our PPO policy and the target imaged by the satellite. Although the policy does image a target 54% of the total time steps, about 75% of those targets were not the intended ones selected by the policy. There are some inherent behaviors of BSK-RL that cause a mismatch between selected and imaged targets including:

- Small overlapping access windows between nearby cities can cause the simulation to lock and slew to a neighboring city if acquired before the policy-prioritized city
- Carry-over imaging from previous maneuvers, when completed in the same step as a new action
- Opportunity window list updating between selection and execution of actions

These issues cause the satellite to earn some reward; however they limit the policies ability to truly optimize its mission. This misalignment indicates the policy intent does not perfectly match simulation execution.

There are also some limits on training that prevent our policy from fully capturing the complex satellite behavior. Our actor-critic network architecture was limited to two 64-node layers. This was sufficient to demonstrate some level of learning, but may not be adequate to fully capture an optimal policy for the scheduling problem. A larger network would lead to additional training time and more resources devoted to the problem.

6) *Summary*: Overall, the PPO-policy improves upon the random and heuristic policies in ways that are clear and consistent. The heuristic policy demonstrates that a basic priority driven behavior is insufficient for this problem due to the long term trade offs between collection opportunities and resource constraints. Although there is a mismatch between the selected and imaged targets, the PPO policy is able to balance these two and provide a strategy that results in higher total reward, more target coverage, higher efficiency, and reduced failure rates. These results demonstrate the potential of RL-based policies to replace heuristic scheduling approaches in constrained spacecraft operations.

VI. CONCLUSION

A. Summary

This work demonstrated the application of a Proximal Policy Optimization (PPO) framework to the problem of autonomous satellite scheduling in a high-fidelity simulation environment. By formulating this task as a POMDP and by integrating realistic spacecraft constraints, we showed that RL can produce policies capable of balancing mission objectives with resource management.

The trained PPO agent consistently outperformed both random and heuristic policies across key performance metrics including total accumulated reward, number of unique targets imaged, and episode duration. These results show that the learned strategy captures non-myopic decision-making behavior. This enables the tradeoff between immediate reward and long term mission goals.

B. Future Work

Future improvements for this exercise would include implementation of level 3 objectives, which were omitted due to time/complexity of implementation. It could also include a larger neural network to expand the learning capabilities of the PPO training. More evaluation episodes would also be required to establish a statistically significant difference of mean values for the PPO and heuristic policies (≈ 500 episodes for Welch's t-test). Additionally, further fidelity to a real spacecraft environment could be added including other subsystems and mission objectives. Finally, the discrepancy between the selected and imaged targets could be improved upon for all future scenarios.

VII. CONTRIBUTIONS AND RELEASE

The authors of this paper grant permission for this work to be posted publicly. The bulk of the simulation work was completed by Michael Plano. The PPO algorithm was implemented by Logan Walthers. The evaluation function for the completed, trained policy was contributed to by both members, which produced most of the figures contained in this work. This report was co-written and reviewed by both members of the team.

REFERENCES

- [1] M. Stephenson and H. Schaub, “Bsk-rl: Modular, high-fidelity reinforcement learning environments for spacecraft tasking,” in *Proceedings of the Conference*, 2024, pp. 1186–1197.
- [2] A. Herrmann and H. Schaub, “A comparative analysis of reinforcement learning algorithms for earth-observing satellite scheduling,” *Frontiers in Space Technologies*, vol. 4, p. 1263489, 2023.
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [4] K. Naik, O. Chang, and C. Kotulak, “Deep reinforcement learning for autonomous satellite responsiveness to observed events,” in *2024 IEEE Aerospace Conference*, 2024, pp. 1–10.
- [5] Y. He, L. Xing, Y. Chen, W. Pedrycz, L. Wang, and G. Wu, “A generic markov decision process model and reinforcement learning method for scheduling agile earth observation satellites,” *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 52, no. 3, pp. 1463–1474, 2022.
- [6] J. Lam, F. Rivest, and J. Berger, “Deep reinforcement learning for multi-satellite collection scheduling,” 12 2019.
- [7] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, “Trust region policy optimization,” *CoRR*, vol. abs/1502.05477, 2015. [Online]. Available: <http://arxiv.org/abs/1502.05477>
- [8] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” in *International Conference on Learning Representations (ICLR)*, 2016, arXiv:1506.02438.
- [9] P. W. Kenneally, S. Piggott, and H. Schaub, “Basilisk: A flexible, scalable and modular astrodynamics simulation framework,” *Journal of Aerospace Information Systems*, vol. 17, no. 9, pp. 496–507, 2020. [Online]. Available: <https://doi.org/10.2514/1.1010762>
- [10] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.